

LE03-ED- Pilhas e filas em C

Parte 1: Questões de Múltipla Escolha

Questão 1 (Pilhas): Uma estrutura de dados do tipo Pilha é governada pelo princípio LIFO (Last-In, First-Out). Considerando uma pilha inicialmente vazia, realizamos sequencialmente as seguintes operações: push(10), push(20), pop(), push(30), push(40), pop(), push(50). Qual é o elemento que reside atualmente no topo da pilha e qual é o total de elementos remanescentes?

- A) O topo é 50 e a pilha possui 3 elementos.
- B) O topo é 50 e a pilha possui 4 elementos.
- C) O topo é 40 e a pilha possui 3 elementos.
- D) O topo Bullseye é 30 e a pilha possui 2 elementos.
- E) O topo é 10 e a pilha está cheia.

Questão 2 (Pilhas): Em uma implementação estática de pilha baseada em array em C, a variável 'topo' é tipicamente inicializada com o valor -1. Qual é a razão técnica para essa escolha de design?

- A) Para permitir que valores negativos sejam armazenados na base da pilha de dados.
- B) Porque em C as strings terminam com -1 em vez do caractere nulo '\0'.
- C) Para indicar que a pilha está vazia, já que o primeiro índice válido de um array em C é 0, permitindo o uso seguro do operador de pré-incremento (++topo) antes da inserção.
- D) Para forçar um erro de Stack Underflow imediatamente na primeira chamada da função push().
- E) Para alocar memória dinamicamente no Stack Pointer do microprocessador.

Questão 3 (Pilhas): Considere o algoritmo clássico de 'Validador de Parênteses' usando pilha apresentado na listagem teórica. Se passarmos a expressão string '((a+b)*c)' e a expressão '((a+b)' para a função validar(), quais serão os retornos lógicos respectivos?

- A) 0 (inválida) e 0 (inválida).
- B) 1 (válida) e 1 (válida).
- C) 0 (inválida) e 1 (válida).
- D) 1 (válida) e 0 (inválida).
- E) Ambas causarão um erro fatal de Stack Overflow.

Questão 4 (Pilhas): A operação clássica 'peek()' (ou top) implementada em uma estrutura de dados abstrata do tipo Pilha realiza qual comportamento?

- A) Remove e retorna obrigatoriamente o elemento posicionado na base da pilha.
- B) Retorna o elemento localizado no topo da pilha, sem removê-lo ou alterar o estado da estrutura.
- C) Limpa completamente a memória de todos os nós alocados.
- D) Inverte a ordem sequencial dos dados usando um mecanismo dinâmico.
- E) Verifica se houve transbordo de memória física (Stack Overflow).

Questão 5 (Pilhas): Qual das seguintes alternativas NÃO representa uma aplicação real e clássica de pilhas na ciência da computação ou na indústria de software?

- A) Mecanismo de 'Desfazer/Refazer' (Undo/Redo - Ctrl+Z) em editores de texto.
- B) Histórico de navegação de navegadores web (para funcionamento do botão 'Voltar').
- C) Resolução de expressões aritméticas e controle de chamadas de funções em compiladores (Call Stack).
- D) Algoritmos de busca que exigem comportamento de caminhamento em profundidade (Backtracking).
- E) Escalonamento de processos de CPU por ordem estrita de chegada de requisições de prioridade igual.

Questão 6 (Filas): Uma Fila (Queue) opera sob o princípio FIFO (First-In, First-Out). Imagine uma fila estática vazia com capacidade para 5 elementos. Executamos as seguintes operações: enqueue(10), enqueue(20), enqueue(30), dequeue(), enqueue(40), dequeue(). Quais elementos permanecem na fila e quem está na frente (head)?

- A) Elementos: [10, 20, 30, 40]; Frente: 10.
- B) Elementos: [30, 40]; Frente: 30.
- C) Elementos: [20, 30, 40]; Frente: 20.

D) Elementos: [30, 40]; Frente: 40.

E) Elementos: [10, 40]; Frente: 10.

Questão 7 (Filas): Qual é o principal problema de usar uma estrutura de Fila Estática Linear simples (onde o índice 'início' apenas avança a cada dequeue) sem reutilização circular de índices, e como ele é resolvido de forma otimizada?

A) Ocorre estouro de pilha (Stack Overflow); resolve-se com malloc().

B) Desperdício de memória, pois posições iniciais liberadas tornam-se inutilizáveis; resolve-se com o Buffer Circular usando o operador de módulo (%).

C) Acesso sequencial lento; resolve-se transformando em árvore binária.

D) O ponteiro 'fim' gera loops infinitos; resolve-se com a função free().

E) Inversão indesejada de dados; resolve-se aplicando o algoritmo LIFO.

Questão 8 (Filas): Em uma estrutura linear clássica de Fila baseada em Listas Encadeadas Dinâmicas, quantos ponteiros externos de controle a estrutura principal precisa gerenciar para manter operações de inserção (enqueue) e remoção (dequeue) com complexidade de tempo constante $O(1)$?

A) Apenas um ponteiro para o elemento central da lista.

B) Nenhum, pois a memória de C faz o gerenciamento de forma automatizada e transparente.

C) Três ponteiros: topo, base e intermediário.

D) Dois ponteiros: um apontando para o início (frente) para remoções eficientes, e outro apontando para o fim (final/cauda) para inserções imediatas.

E) Um único ponteiro nulo para tratar o Underflow.

Questão 9 (Filas): Dentre os cenários computacionais abaixo, assinale aquele que descreve a aplicação ideal e direta de uma estrutura de Fila (Queue):

A) Um interpretador de comandos que precisa desfazer as últimas ações do usuário corporativo.

B) Um spooler de impressão de rede que gerencia arquivos enviados concorrentemente por múltiplos usuários para uma única impressora de escritório.

C) O sistema de gerenciamento de chamadas recursivas de um software matemático complexo.

D) A verificação sintática e balanceamento de tags HTML/XML em um navegador.

E) O armazenamento temporário dos dados locais de um struct Pessoa para passagem por valor.

Questão 10 (Filas): Considerando o quadro comparativo apresentado entre Pilhas e Filas no material teórico de suporte, qual das seguintes afirmações estabelece uma diferença correta de controle de ponteiros entre as duas estruturas?

A) A pilha exige dois ponteiros de controle móveis, enquanto a fila opera com apenas um ponteiro fixo no centro.

B) Na pilha, a inserção e exclusão ocorrem em extremidades opostas; na fila ocorrem na mesma ponta.

C) A pilha necessita de apenas um ponteiro para rastreamento (o topo), enquanto a fila necessita de dois indicadores (início e fim) para gerenciar as duas extremidades ativas.

D) Ambas utilizam exatamente a mesma quantidade de ponteiros, diferenciando-se apenas pelo uso do operador de módulo aritmético (%).

E) Nenhuma das estruturas aceita ponteiros dinâmicos obtidos através de funções como malloc() ou calloc().

Parte 2: Análise de Código e Correção de Erros

Instruções: Esta seção apresenta códigos de laboratório contendo falhas graves de lógica, sintaxe ou gerenciamento de memória em C, seguidos por suas respectivas correções detalhadas e justificadas para avaliação.

2.1) Script Errado 1: Script com Erro sobre Pilha (61Exemplo_Bug.c)

```
#include <stdio.h>
#define MAX 5

int pilha[MAX];
int topo = -1;

int pop() {
```

```
// ERRO GRAVE: Falta validação de subfluxo (Pilha Vazia)
// Se o topo for -1, tentará acessar pilha[-1] gerando comportamento indefinido!
return pilha[topo--];
}

int main() {
    printf("Desempilhando: %d\n", pop());
    return 0;
}
```

Listagem 1.1: Código com Bug**Lista de Erros Técnicos (Para o Professor):****Listagem 1.2: código correto****2.2) Cenário 2: Script com Erro sobre Pilha Dinâmica (62Exemplo_Bug.c)**

```
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No* proximo;
} No;

No* topo = NULL;

void push(int valor) {
    No* novo = (No*)malloc(sizeof(No));
    novo->dado = valor;
    novo->proximo = topo;
    topo = novo;
}

int pop() {
    if (topo == NULL) return -1;
    int valor = topo->dado;
    topo = topo->proximo; // ERRO GRAVE: O nó antigo foi desconectado mas continua alocado na
    // memória RAM! (Memory Leak)
    return valor;
}

int main() {
    push(10); pop(); // Perda de ponteiro de referência sem desalocação
    return 0;
}
```

Listagem 2.1: código com bug**Lista de Erros Técnicos (Para o Professor):**

- 1) ???
- 2) ???
- 3) ???

Listagem 2.2: código correto**2.3) Cenário 3: Script com Erro sobre Fila Linear (66Fila_Bug.c)**

```
#include <stdio.h>
#define MAX 3

int fila[MAX];
int inicio = 0, fim = -1, total = 0;

void enqueue(int v) {
    if (total >= MAX) {
        printf("Fila Cheia!\n");
        return;
    }
    fila[++fim] = v; // ERRO: Inserção estritamente linear sem dar a volta (Aritmética Não Circular)
    total++;
}

int dequeue() {
    if (total == 0) return -1;
    total--;
    return fila[inicio++];
}

int main() {
    enqueue(10); enqueue(20); enqueue(30);
    dequeue(); // Libera uma vaga física no índice 0
    enqueue(40); // ERRO: fim tentará acessar fila[3], que está fora dos limites (Overflow de Array)!
    return 0;
}
```

Listagem 3.1: código com erro**Lista de Erros Técnicos (Para o Professor):**

- 4) ???
- 5) ???
- 6) ???

Listagem 3.2: código correto**2.4) Cenário 4: Script com Erro sobre Fila Encadeada Dinâmica (67Fila_Bug.c)**

```
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No* prox;
} No;

No* inicio = NULL;
No* fim = NULL;

void dequeue_errado() {
    // ERRO DE LÓGICA E MEMÓRIA
    if (inicio == NULL) return;
    if (inicio == fim) {
        inicio = NULL; // ERRO: fim continua apontando para o bloco que será perdido
    } else {
        inicio = inicio->prox;
    }
    // Nota: O nó removido nunca é liberado usando free()!
}
```

Listagem 4.1: código com erro**Lista de Erros Técnicos (Para o Professor):**

- 7) ???
- 8) ???
- 9) ???

Listagem 4.3: código correto